— Solution notes —

COMPUTER SCIENCE TRIPOS Part IB – 2024 – Paper 5

4 Concurrent and Distributed Systems (djg11)

Semaphores Monitors

(*a*) A concurrency library provides a `signal` primitive for semaphores and a `notify` primitive for condition variables. Explain what they have in common and how they are different. Are they blocking primitives? Can they have no effect? [4 marks]

Answer: Both potentially enable another thread to start running. The semaphore operator will always have an effect, either incrementing the semaphore or making a thread ready to run. When a condition variable is notified, there may be no thread queued on it, so there will be no effect. Neither primitive is intrinsically blocking (ignoring Hoare-style monitors, that were deprecated in lectures), [A side effect of these primitives can be to change what is most worthy of running, leading at some point to the issuer's preemption if there is a shortage of processors, but that is not the primitive itself blocking.]

Isolation Serialisable

(*b*) A transaction processing system uses non-strict isolation.

(*i*) Without using an example, define the notion of serialisability of transactions. [2 marks]

Answer: The post-hoc aggregate effect of a set of transactions on shared state is serialisable if it is equal to outcome of some particular, isolated, serial ordering of those transactions. [NB. It is the overall effect of the transactions, not the transactions themselves, that is serialisable.]

(*ii*) What is a conflict between two threads, and can conflicts be used to check for serialisability? [2 marks]

Answer: A potential conflict arises when two transaction threads operate on a common resource. The simple definition lectured is that a conflict arises if the operations cannot be swapped over in the time domain. RaR is not a conflict, but WaR, WaW and RaW are conflicts.

Isolation

(*iii*) What are the advantages and disadvantages of non-strict isolation? [2 marks]

Answer: Strict isolation avoids cascading aborts but reduces allowable concurrency. A further short sentence explaining at least one of these terms is needed for full credit. [NB: Strict isolation does not stop transactions that overlap in the time domain from operating on common resources: eg. 2PL.]

(*iv*) Can performance be enhanced when increment and decrement operations are considered as composite operations? Consider this in the context of conflict analysis techniques. [2 marks]

Answer: A sequence of atomic increment/decrement operation can be permuted in time, if not interleaved with other operations on those objects, whether performed by

one or several threads. A necessary side condition is that the underlying resource does not saturate (ie. all operations return successfully). For example, a debit on a bank account may fail if the overdraft limit is reached. [The same applies for various other operations such as bitset, but here the idempotent or saturating nature of the effected resource helps.]

If an increment/decrement is instead implemented as part of a composite sequence, minimally including a load, add a store, a greater number of interleaving of operations is possible, removing the atomic advantage. This is unlikely to improve performance.

Owing perhaps to the word atomic *not being used in the question, some candidates confused* atomic *and* composite, *but full credit was awarded when it was clear they understood the concepts.*

(*v*) Give an example, not using increments or decrements, where simplistic conflict analysis may report a problem that in reality is not a problem. [2 marks]

Answer: An example is a WaW situation where both transactions happen to store the same value. Hence the two writes can be seen to not conflict if the analysis takes the data values into account. [NB. An account debit or credit is an increment/decrement and is not an acceptable answer.]

Reliable message passing

(*c*) Some code is replaced in an asynchronous, reliable message-passing language that uses `channel_id!value` and `?channel_id` to communicate between user-space threads. The whole system has now seized up. Sketch two possible code fragments that may have caused the seizure, one due to deadlock, and one not due to deadlock. [6 marks]

Answer: In reliable message passing, a sending process blocks when writing to a full output queue. Equally, a receive operator, such as `let y = cosine(?foo)` will block while channel `foo` is empty.

The pattern of dependencies between the relevant processes can be linear or cyclic (ie. they are both inside a strongly-connected component of the communication graph or not). A cyclic dependency is required for the seize to be textbook deadlock.

```
P1: let x = ?foo in bar!(x+11);       P2: let y = ?bar in foo!(y+21);
```

Provided no other process writes or reads on the named channels, P1 and P2 are deadlocked.

An example without a cycle: The system seizes when a critical source is blocked on a send because a sink has been changed to never be ready, or vice versa. This is the 'reliable multicast' problem. For instance, a 'master clock' process C1 will stop if one of its output buffers fills up, e.g. owing to child code in C2 going indefinitely busy (predicate `pf` never returning false).

```
C1: while (true) { foo!"clock";  foo!"clock";  foo!"clock";  ... }
C2: while (true) { let _ = ?foo; while (pf(...)) {  ...code not reading foo... }}
```

This third example livelocks, which is not deadlock. A black-box view of something that internally suffers livelock can appear as a seizure.

```
L1: { baz!10; while (?bar != 11) { baz!33 }; ... useful behaviour ... }
L2: { while (?baz != 12) { bar!34 }; ... further useful behaviour ... }
```

(An even simpler livelock is all thread going into infinite loops with no channel activity.)
